

Randomized Quick Sort

Probabilistic Algorithms & Expected-Time Analysis

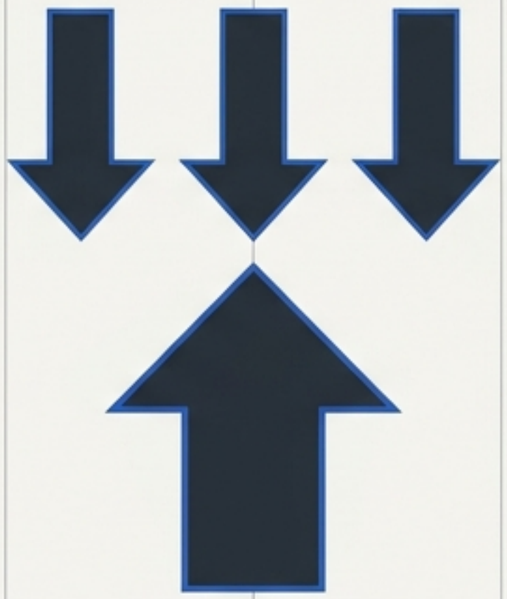
An AI-assisted educational recreation based on MIT OpenCourseWare:
6.046J / 18.410J Design and Analysis of Algorithms (Spring 2015).

Gemini Notebook

Divide-and-Conquer and Memory Usage

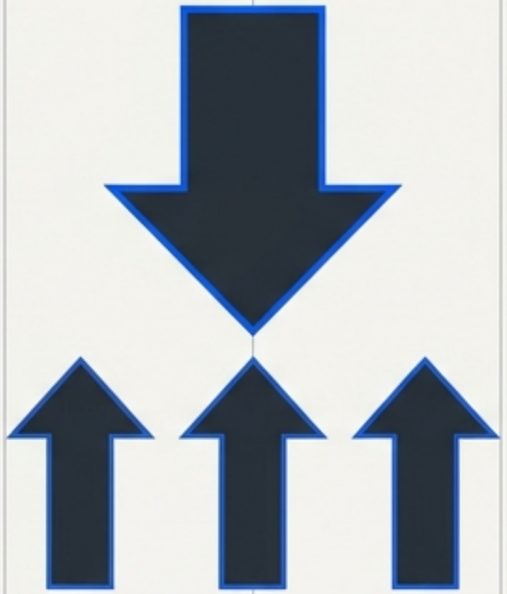
Quicksort is a classic divide-and-conquer algorithm, but it inverts the typical workload: almost all heavy lifting happens during the divide step, making the combine step trivial.

Mergesort



- Workload: Heavy during Combine phase.
- Memory Constraint: Requires $O(n)$ auxiliary space to merge.

Quick Sort



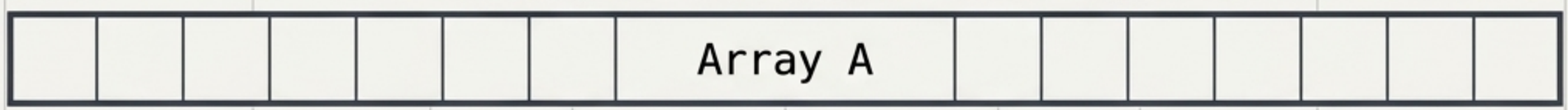
- Workload: Heavy during Divide phase.
- Memory Advantage: Sorts in-place (like insertion sort), requiring only $O(1)$ auxiliary space.

Gemini Notebook

Pivot Partitioning: L, E, and G

The core of Quicksort is the partition step, which runs in $O(n)$ time.
Each insertion and removal takes $O(1)$ time.

Divide: Pick a pivot element x from array A.



Partition: Rearrange A into three distinct sub-arrays:



All elements $< x$ | All elements $= x$ | All elements $> x$

Conquer: Recursively sort sub-arrays L and G.

Gemini Notebook

Illustrative example; not a direct quote from source material.

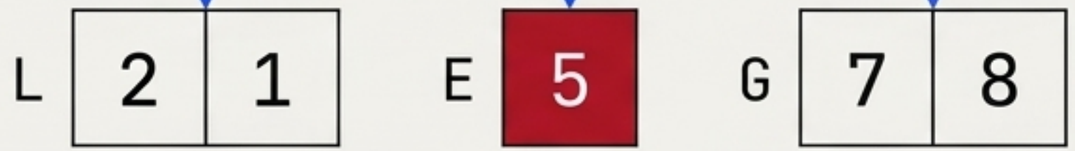
Partitioning in Action

Step 1: Initial Array



Choose pivot x = 5

Step 2: First Partition



Step 3: Recursive Calls

Recurse on L: [2, 1] → pivot x = 2

L: [1], E: 2], G: [ ]

Recurse on G: [7, 8] → pivot x = 7

L: [ ], E: 7], G: [8]

Step 4: Combine

Trivial reassembly:



Gemini Notebook

Deterministic vs. Randomized Pivot Selection



Basic Quicksort (Deterministic)

- **Mechanism:** Always pivots around $x = A[1]$ or $A[n]$.
- **The Flaw:** If the input is already sorted (or reverse sorted), one sub-array gets $n - 1$ elements and the other gets 0.
- **Result:** Disastrous worst-case running time of $\Theta(n^2)$.



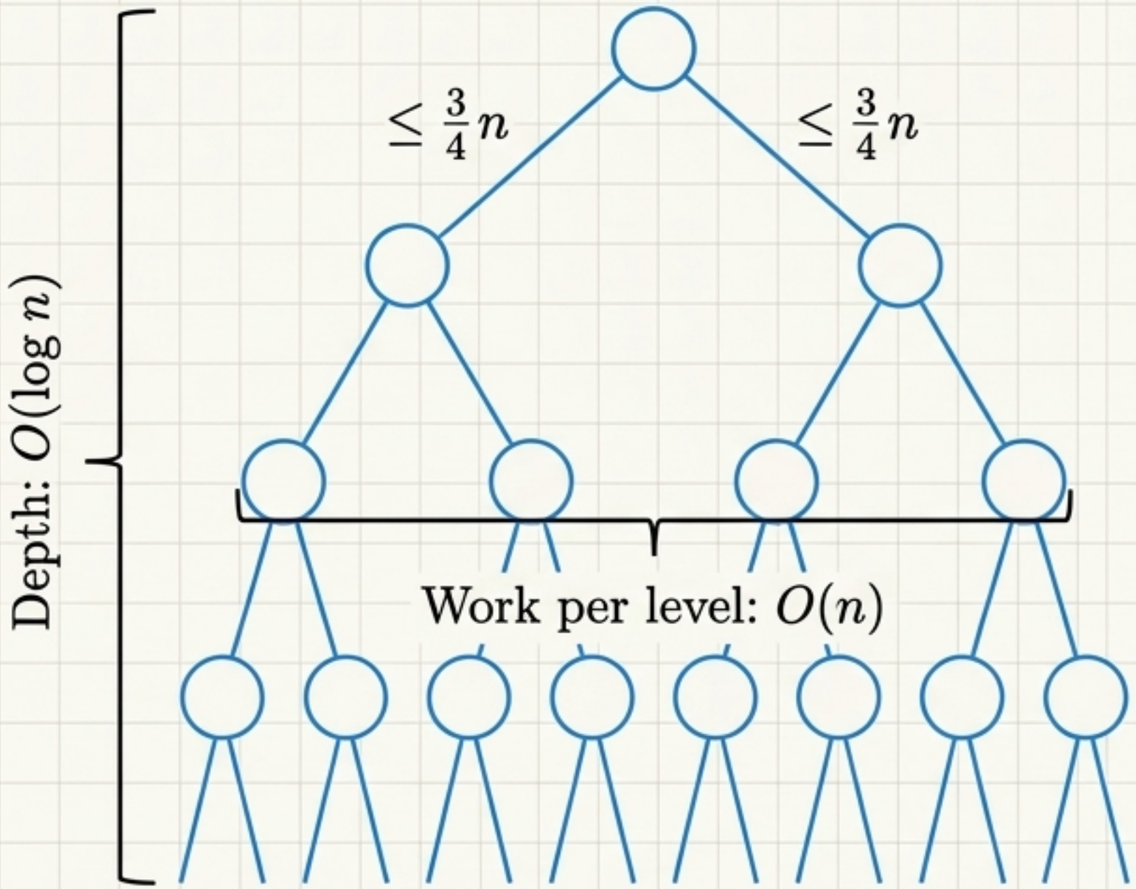
Randomized Quicksort (Las Vegas Algorithm)

- **Mechanism:** Pivot x is chosen uniformly at random at each recursive step.
- **The Fix:** A Las Vegas algorithm that neutralizes specific worst-case input configurations; always correct, efficient in expectation.
- **Result:** Expected time is $O(n \log n)$ for all input arrays.

Gemini Notebook

Expected-Time Analysis: Paranoid Quick Sort

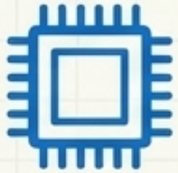
Paranoid Quick Sort mathematically guarantees balanced splits by repeating the random pivot selection until $|L| \leq (3/4)|A|$ and $|G| \leq (3/4)|A|$.



- **Good Pivots:** A randomly chosen pivot meets the $\frac{3}{4}$ criteria with probability $> 1/2$.
- **Iterations:** Expected iterations to find a good pivot is ≤ 2.
- **Tree Depth:** Because sub-arrays are at most $(\frac{3}{4}4)n$, the maximum depth of the recursion tree is strictly bounded at $O(\log n)$.
- **Level Work:** At each level of the tree, the algorithm performs $O(n)$ work to partition elements.
- **Total Expected Runtime:** $O(\log n)$ depth $\times$ $O(n)$ work $=$ Expected $O(n \log n)$ time.

Gemini Notebook

Key Takeaways



In-Place Efficiency

Quicksort is a powerful divide-and-conquer strategy that sorts in-place, bypassing the $O(n)$ memory cost of alternatives like Mergesort.



The Power of Randomization

By functioning as a Las Vegas algorithm, randomized pivot selection breaks deterministic worst-case input vulnerabilities, ensuring expected $O(n \log n)$ performance across the board.



Analytical Rigor

Paranoid Quick Sort provides an elegant analytical framework, proving that bounded splits ($O(\log n)$ depth $\times$ $O(n)$ level work) mathematically guarantee optimal expected runtimes.

Gemini Notebook